



S.T.U.S. Mandal's
Sangola Mahavidyalya, Sangola

PROJECT REPORT ON

"MYSQL MANAGER"

Web-Based MySQL Database Management System

Submitted By:

Mr. Aniket Gavali
&
Mr. Rohan Sutar

Under the Guidance of:

Prof. Dr. Tathe R.R.
Prof. Pawar K.B.

Submitted to

Punyashlok Ahilyadevi Holkar University, Solapur

In Practical Fulfilment of the

Bachelor Degree of Computer Science
(Entire Computer Science) — III
(2025-2026)



S. T. U. S. Mandal's

Sangola Mahavidyalaya, Sangola

CERTIFICATE

Roll No. :4127

Roll No. :4190

This is to certify that, Mr. Aniket Gavali and Mr. Rohan Sutar have successfully completed the project titled "MYSQL MANAGER — Web-Based MySQL Database Management System" at Sangola Mahavidyalaya, Sangola under the guidance and supervision of Prof. Dr. Tathe R.R. and Prof. Pawar K.B. in fulfilment of requirements of final year, Bachelor of Computer Science (Entire Computer Science) of working year 2025-2026.

Place: Sangola

Date: / /2026

Seen By
Prof. Pawar K.B.
(Project Guide)

H. O. D.
Prof. Dr. Tathe R.R.
(Dept. of Computer Science)

External Examiner

DECLARATION

To,
The Principal,
Sangola Mahavidyalaya, Sangola.

Respected Sir/Madam,

We, hereby declare that the project titled "**MYSQL MANAGER**" is an original work developed under the guidance of Prof. Dr. Tathe R.R. and Prof. Pawar K.B. at Sangola Mahavidyalaya, Sangola. All sources of information used in this report have been duly acknowledged.

We further declare that we have not copied this project from any other project submitted to Punyashlok Ahilyadevi Holkar University, Solapur, or any other university or institution earlier.

Place: Sangola

Date: / /2026

Mr. Aniket Gavali
(Student)

Mr. Rohan Sutar
(Student)

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Department of Computer Science and all the faculty members of Sangola Mahavidyalaya, Sangola for their invaluable support and encouragement throughout the successful completion of this project.

We are deeply thankful to Prof. Pawar K.B. and Prof. Dr. Tathe R.R. for their expert guidance, consistent support, and motivation during the development of "MySQL Manager". Their technical insights into Django web development and database management were absolutely crucial in shaping this project.

We also thank the Principal of Sangola Mahavidyalaya, Sangola for providing the infrastructure and resources needed for this project. Special thanks to our classmates for their feedback during testing and to our families for their patience and constant encouragement.

Mr. Aniket Gavali
(Student)

Mr. Rohan Sutar
(Student)

INDEX

Sr. No.	Chapter / Topic	Page No.
1	Introduction	6
2	Requirements	8
3	System Architecture	10
4	Data Flow Diagram -DFD	12
4	Source Code	15
5	Output Screens	46
6	Testing Approach	48
7	Features & Enhancements	48
8	Conclusion	49
9	Future Scope	51
10	Bibliography	52
11	Thank You	52

1. INTRODUCTION

1.1 Overview

MySQL Manager is a Django-based web application that provides users with a browser-based interface to manage MySQL databases without using the command line. The project addresses a real need: database administrators and developers frequently need to perform routine database operations — creating databases, managing tables, inserting and viewing records — and a graphical web interface dramatically reduces the complexity and risk of mistakes compared to raw SQL command-line tools.

The application is built on Python's Django framework, leveraging its powerful ORM, authentication system, and URL routing. The frontend uses HTML, CSS, JavaScript, and Tailwind CSS for a clean, responsive interface. Git and GitHub were used for version control throughout the development process, as evidenced by the commit history spanning features such as login, database creation, table management, record viewing, and UI improvements.

The system was developed as a final-year academic project by Mr. Aniket Gavali and Mr. Rohan Sutar, demonstrating practical skills in full-stack web development, database administration, and secure backend design.

1.2 Motivation

Managing MySQL databases via the command-line interface (CLI) requires memorising complex SQL syntax, is prone to human error, and is inaccessible to users who are not experienced with SQL. Tools like phpMyAdmin exist but are heavyweight, require Apache/PHP, and have a complex configuration process. The motivation behind MySQL Manager was to build a lightweight, Django-native, web-based alternative that is easy to deploy, visually clean, and focused specifically on the most common database management tasks.

The project also served as an opportunity for Mr. Aniket Gavali and Mr. Rohan Sutar to gain deep practical experience with Django's MVT (Model-View-Template) architecture, dynamic MySQL connectivity, session-based authentication, and modern frontend design with Tailwind CSS.

1.3 Objectives

- To develop a secure web-based interface for managing MySQL databases without using the command line.
- To implement Django-based user authentication with session management for secure access.
- To enable users to create new MySQL databases dynamically through the web interface.
- To provide a database explorer that lists all available databases and their tables.
- To allow users to create, view, and delete tables within a selected database.
- To display table records in a structured, paginated format.
- To implement toast notifications for user feedback on all database operations.
- To build a visually appealing, responsive interface using Tailwind CSS with a custom cursor.
- To use Git and GitHub for version control and collaborative development.

1.4 Scope of the Project

The scope of the MySQL Manager system covers the following functional areas:

- Secure login and logout with Django's built-in authentication framework.
- Dashboard showing all available MySQL databases accessible to the connected user.
- Database creation module with name input and immediate feedback.
- Database explorer ("Work" and "Explore" sections as referenced in commit history).
- Table listing within a selected database.
- Table creation with column definitions.
- Table deletion with confirmation.
- Record viewing for any selected table.
- Responsive frontend with custom cursor and Tailwind CSS utility styling.

1.5 Problem Statement

Database administrators and developers face the following challenges when working with MySQL from the command line:

- Syntax complexity: MySQL CLI requires precise SQL syntax; a single typo can cause errors or data loss.
- Accessibility barrier: Non-technical users and beginners cannot effectively use the MySQL CLI.
- No visual feedback: CLI gives minimal visual cues about database structure and contents.
- Remote management difficulty: SSH-based CLI management of remote databases is cumbersome.
- Lack of lightweight alternatives: Existing tools like phpMyAdmin are heavy and require complex LAMP stack setup.

MySQL Manager solves all of these by providing an intuitive, Django-powered web application accessible from any browser, secured with session authentication, and deployable with a simple Python environment.

2. REQUIREMENTS

2.1 Functional Requirements

- **User Authentication:** The system shall provide login and logout functionality using Django's built-in authentication framework. Only authenticated users shall access the database management features.
- **Database Listing:** The system shall display all MySQL databases accessible to the configured database user on the dashboard.
- **Database Creation:** The system shall allow authenticated users to create new MySQL databases by entering a database name and submitting the form.
- **Database Exploration:** The system shall provide a browsable explorer showing tables within a selected database.
- **Table Listing:** The system shall list all tables within any selected database.
- **Table Creation:** The system shall allow users to define and create new tables with column names and data types.
- **Table Deletion:** The system shall allow users to delete selected tables with a confirmation step and toast notification.
- **Record Viewing:** The system shall display the contents of any selected table in a structured, readable format.
- **Toast Notifications:** The system shall display toast notifications for all major operations (table created, table deleted, database created, errors).
- **Custom Cursor:** The system shall implement a custom cursor for enhanced user experience.

2.2 Non-Functional Requirements

2.2.1 Security

All routes except the login page shall require authentication. Django's CSRF protection shall be enabled for all form submissions. Database credentials shall be stored in Django settings and not exposed to the frontend. The application shall use Django session tokens for maintaining login state.

2.2.2 Performance

All page loads shall complete within three seconds under normal conditions. Database queries shall use efficient SQL commands directly via Django's database connection. The Tailwind CSS shall be loaded via CDN to reduce build complexity.

2.2.3 Usability

The interface shall be intuitive and require no SQL knowledge from the user. All actions shall provide immediate feedback via toast notifications. The layout shall be consistent across all pages with a persistent navigation bar.

2.2.4 Responsiveness

The interface shall adapt to different screen sizes using Tailwind CSS responsive breakpoints. Navigation and data tables shall be functional on both desktop and mobile viewports.

2.2.5 Maintainability

The codebase shall follow Django's MVT pattern with clear separation of models, views, templates, and URL routing. Version control shall be maintained using Git with descriptive commit messages.

2.3 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 / AMD equivalent (1.6 GHz+)
RAM	4 GB (8 GB recommended)
Storage	1 GB free disk space
Network	Localhost (no internet required after setup)
Display	1280 x 720 resolution or higher

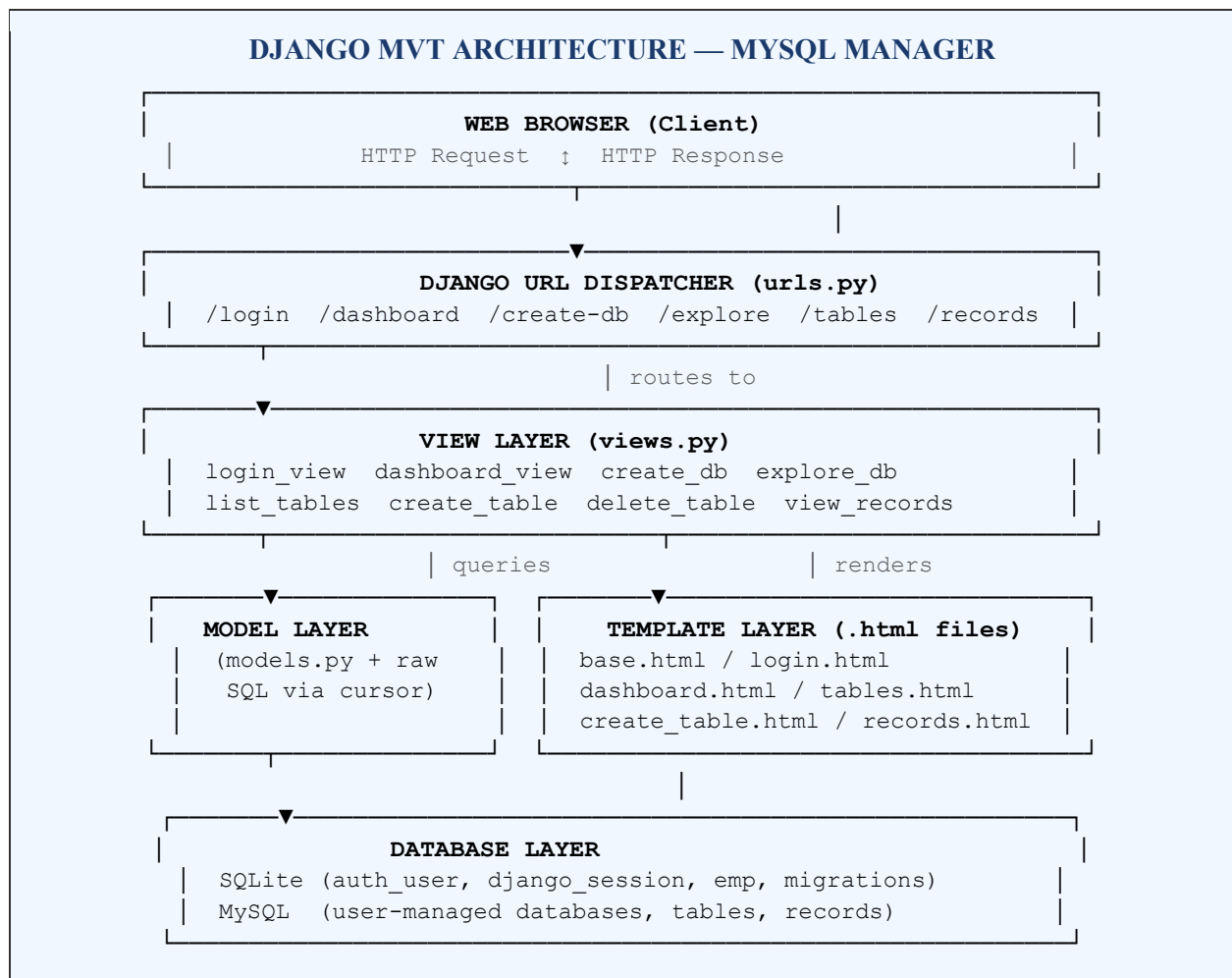
2.4 Software Requirements

Software / Technology	Version / Details
Operating System	Windows 10/11, macOS 11+, or Ubuntu 20+
Python	v3.9 or higher
Django	v4.x (Web Framework)
MySQL	v8.0 Community Edition
mysqlclient	Python MySQL adapter for Django
Tailwind CSS	v3.x (via CDN)
HTML / CSS / JS	Django template engine
SQLite	Django default DB for auth/session/admin
Git & GitHub	Version control and code hosting
Web Browser	Google Chrome 90+, Firefox 88+, Edge 90+
Code Editor	Visual Studio Code

3. SYSTEM ARCHITECTURE

3.1 Django MVT Architecture

MySQL Manager follows Django's MVT (Model-View-Template) architectural pattern, which is Django's variant of the classic MVC (Model-View-Controller) pattern. In MVT, the Model handles data and database interaction, the View contains the business logic and processes HTTP requests, and the Template renders the HTML response sent to the browser. Django itself acts as the Controller by routing URL requests to the appropriate View.



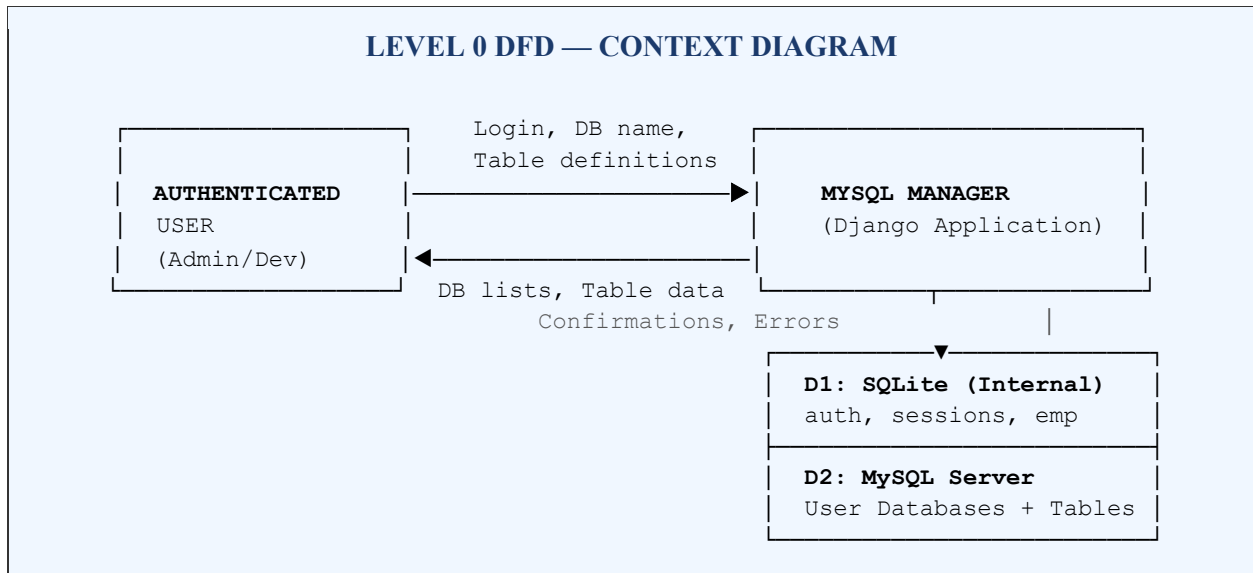
3.3 Project Folder Structure

```
MYSQL-MANAGER/
├── manage.py
├── db.sqlite3
├── README.md
├── .gitignore
├── MYSQL_MANAGER/
│   ├── __init__.py
│   ├── urls.py
│   └── wsgi.py
├── mysql/
│   ├── __pycache__/
│   ├── migrations/
│   │   └── __init__.py
│   ├── templatetags/
│   │   └── __init__.py
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
└── templates/
    ├── create_table.html
    ├── delete_table.html
    ├── edit_table.html
    ├── explore.html
    ├── home.html
    ├── insert_records.html
    ├── login.html
    ├── table_view.html
    └── update_records.html
```

4. DATA FLOW DIAGRAM (DFD)

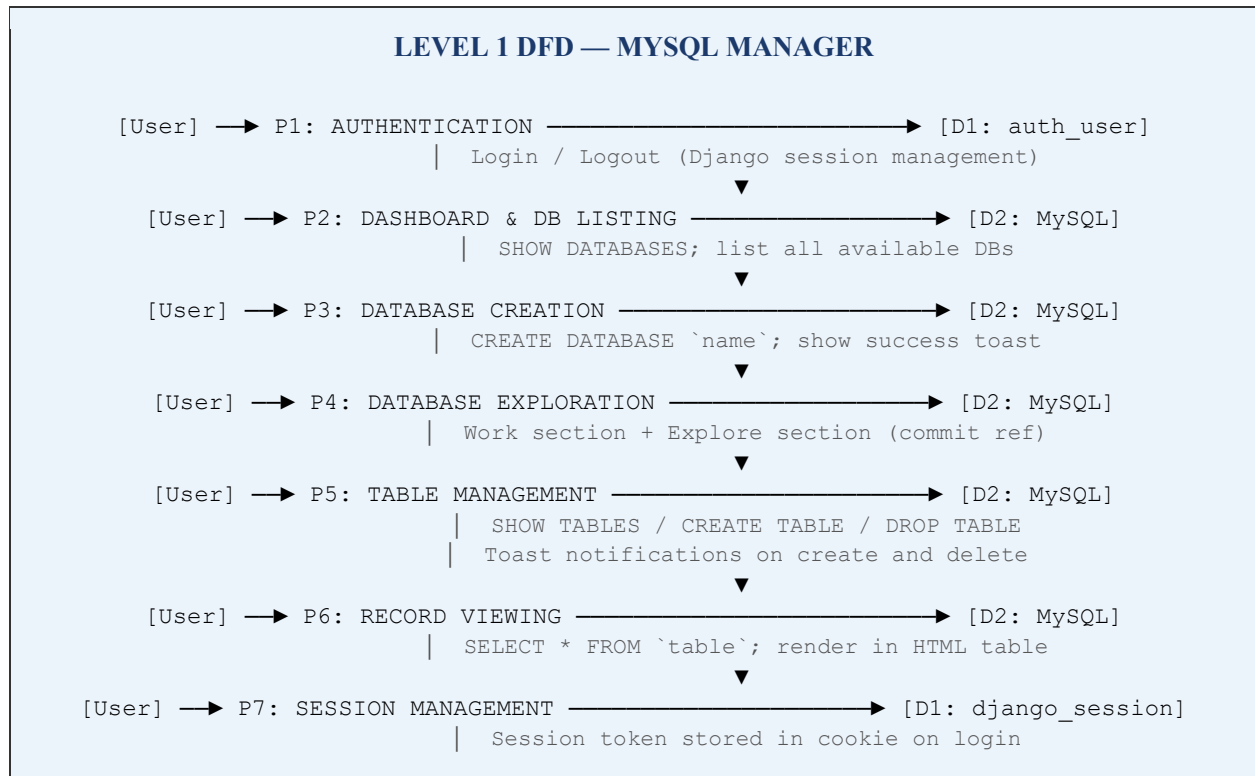
4.1 Level 0 DFD — Context Diagram

The Level 0 DFD presents the MySQL Manager as a single process. The primary external entity is the Authenticated User (database administrator or developer). Data flows between the user's browser and the system, and between the system and two data stores: the SQLite database (for Django internals) and the MySQL server (for user-managed databases).



4.2 Level 1 DFD — Main Processes

The Level 1 DFD decomposes the system into its core processes as developed in the application, referencing the actual commit history (login, modified work/explore section, view and creation, table creation/deletion).

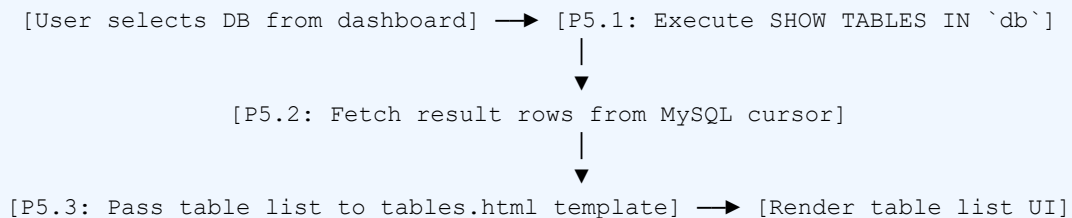


4.3 Level 2 DFD — Table Management Process

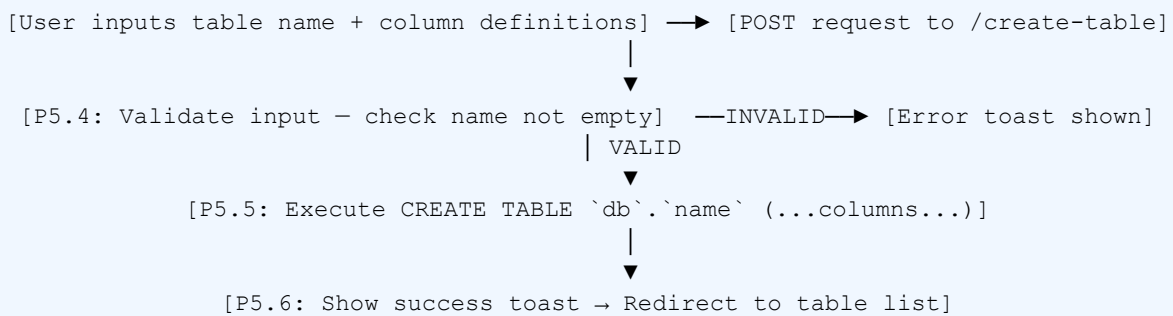
This expands Process P5 (Table Management) which was the most actively developed feature based on the commit history (commits: 'Table creation toast added', 'Table deletion section updated').

LEVEL 2 DFD — P5: TABLE MANAGEMENT

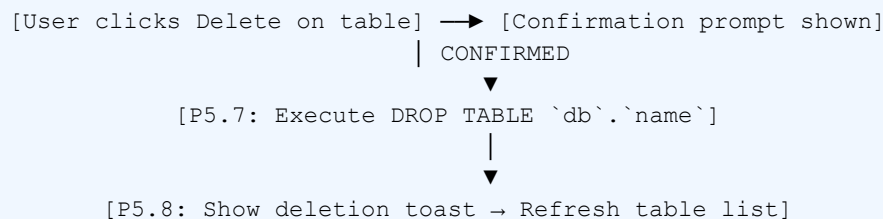
LIST TABLES FLOW:



CREATE TABLE FLOW:



DELETE TABLE FLOW:



5. SOURCE CODE

5.1 manage.py — Django Entry Point

```
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import os
import sys

def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'mysql_manager.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()
```

5.2 settings.py — Database Configuration

```
"""
Django settings for mysql_manager project.

Generated by 'django-admin startproject' using Django 5.2.5.

For more information on this file, see
https://docs.djangoproject.com/en/5.2/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/5.2/ref/settings/
"""

from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-l*r%d)f0%)^4h$f69b(8d=46_bwb!t8^qn_z%+1(pwh35tea2) '

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
```

```
'django.contrib.contenttypes',
'django.contrib.sessions',
'django.contrib.messages',
'django.contrib.staticfiles',
'mysqli',
]

MIDDLEWARE = [
'django.middleware.security.SecurityMiddleware',
'django.contrib.sessions.middleware.SessionMiddleware',
'django.middleware.common.CommonMiddleware',
'django.middleware.csrf.CsrfViewMiddleware',
'django.contrib.auth.middleware.AuthenticationMiddleware',
'django.contrib.messages.middleware.MessageMiddleware',
'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'mysql_manager.urls'

TEMPLATES = [
{
    'BACKEND': 'django.template.backends.django.DjangoTemplates',
    'DIRS': [BASE_DIR, 'templates'],
    'APP_DIRS': True,
    'OPTIONS': {
        'context_processors': [
            'django.template.context_processors.request',
            'django.contrib.auth.context_processors.auth',
            'django.contrib.messages.context_processors.messages',
        ],
    },
},
]

WSGI_APPLICATION = 'mysql_manager.wsgi.application'

# Database
# https://docs.djangoproject.com/en/5.2/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

# Password validation
# https://docs.djangoproject.com/en/5.2/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
{
    'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
},
{
    'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
},
{
    'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
},
{
    'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
},
]
```



```
]

# Internationalization
# https://docs.djangoproject.com/en/5.2/topics/i18n/

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True

# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.2/howto/static-files/

STATIC_URL = 'static/'

# Default primary key field type
# https://docs.djangoproject.com/en/5.2/ref/settings/#default-auto-field

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

5.3 views.py — Core View Functions

```
from django.contrib import messages
from django.db import connection
from django.http import HttpResponse, HttpResponseRedirect
from django.shortcuts import render, redirect
import mysql.connector
from mysql.connector import Error

from django.shortcuts import render, redirect
import mysql.connector
from mysql.connector import Error

def login(request):
    if request.method == "POST":
        pas = request.POST.get("p", "").strip()

        if pas == "":
            return render(request, "login.html", {"field_error": "Password is
required."})

        try:
            conn = mysql.connector.connect(
                host="localhost",
                user="root",
                password=pas
            )

            if conn.is_connected():
                request.session["mysql_password"] = pas
                conn.close()
                return redirect("/home/")

        except Error as e:
            print(e)    # IMPORTANT: show real error in terminal
            return render(request, "login.html", {"error": str(e)})

    return render(request, "login.html")
```

```

def home(request):
    return render(request, 'home.html')

def explore(request):
    return render(request, 'explore.html')

def get_dbs(request):
    try:
        pas = request.session.get("mysql_password")

        conn = mysql.connector.connect(host="localhost", user="root", password=pas)
        cl = conn.cursor()
        q = "SHOW DATABASES"
        cl.execute(q)

        dbs=[]
        for i in cl.fetchall():
            dbs.append(i[0])
        conn.close()

    except Error as e:
        dbs = [f"Error: {e}"]

    return render(request, "explore.html", {"databases": dbs})

def create_dbs(request):
    dbs = []
    message = ""
    try:
        if request.method == "POST":
            pas = request.session.get("mysql_password")
            dbname = request.POST.get("db_name")

            conn = mysql.connector.connect(host="localhost", user="root",
password=pas)
            cl = conn.cursor()

            q = f"CREATE DATABASE {dbname}"
            cl.execute(q)

            message = f"Database '{dbname}' created successfully  "

            cl.execute("SHOW DATABASES")
            dbs = [i[0] for i in cl.fetchall()]

            conn.close()
    except Error as e:
        message = f"Error: {e}"

    return render(request, "explore.html", {"databases": dbs, "message": message})

def delete_dbs(request):
    dbs=[]
    del_message=""

    try:
        if request.method=="POST":
            dbname=request.POST.get("delddb")
            pas =request.session.get("mysql_password")

            conn = mysql.connector.connect(host="localhost", user="root",
password=pas)

```

```

        cl = conn.cursor()

        q=f"drop database {dbname}"
        cl.execute(q)
        del_message = f"Database '{dbname}' Deleted successfully  "

        cl.execute("SHOW DATABASES")
        dbs = [i[0] for i in cl.fetchall()]

        conn.close()
    except Error as e:
        message = f"Error: {e}"

    return render(request, "explore.html", {"databases": dbs, "message": del_message})

def work(request):
    return render(request, 'work.html')

def select_db(request):
    select_db_name = ""
    error_msg = ""

    if request.method == "POST":
        try:
            pas = request.session.get("mysql_password")
            name = request.POST.get("db_name")

            conn =
mysql.connector.connect(host="localhost",user="root",password=pas,database=name)

            if conn.is_connected():
                request.session["current_db"]=name
                select_db_name = name

        except Error as e:
            error_msg = str(e)
            print("MySQL Error:", error_msg)

    context = {"selected_db": select_db_name,"error": error_msg}

    return render(request, 'work.html', context)

def show_tables(request):
    tables = []
    error_msg = None

    try:
        pas = request.session.get("mysql_password")
        db = request.session.get("current_db")

        conn =
mysql.connector.connect(host="localhost",user="root",password=pas,database=db)
        cursor = conn.cursor()
        cursor.execute("SHOW TABLES")
        tables = [table[0] for table in cursor.fetchall()]
        cursor.close()
        conn.close()

    except Error as e:
        error_msg = str(e)
        print("MySQL Error:", error_msg)

    context = {"tables": tables,"selected_db": db,"error": error_msg}

```

```
    return render(request, 'work.html', context)

def view_table(request):
    table_structure = []
    table_data = []
    table_name = ""
    error_msg = None
    selected_db = request.session.get("current_db")

    if request.method == "POST":
        try:
            pas = request.session.get("mysql_password")
            db = request.session.get("current_db")
            table_name = request.POST.get("table_name")
            request.session["selected_table"] = table_name

            if not db:
                error_msg = "No database selected"
                context = {"error": error_msg, "selected_db": selected_db}
                return render(request, 'work.html', context)

            conn = mysql.connector.connect(host="localhost", user="root",
password=pas, database=db)
            cursor = conn.cursor()

            # Get table structure
            cursor.execute(f"DESCRIBE {table_name}")
            structure_result = cursor.fetchall()

            # Format structure data
            for row in structure_result:
                table_structure.append({
                    'field': row[0],
                    'type': row[1],
                    'null': row[2],
                    'key': row[3],
                    'default': row[4] if row[4] is not None else 'NULL',
                    'extra': row[5]
                })

            # Get table data (limit to first 100 rows for performance)
            cursor.execute(f"SELECT * FROM {table_name} LIMIT 100")
            columns = [desc[0] for desc in cursor.description]
            data_result = cursor.fetchall()

            # Format data
            for row in data_result:
                row_dict = {}
                for i, value in enumerate(row):
                    row_dict[columns[i]] = value if value is not None else 'NULL'
                table_data.append(row_dict)

            cursor.close()
            conn.close()

        except Error as e:
            error_msg = str(e)
            print("MySQL Error:", error_msg)

    context = {
        "table_structure": table_structure,
```

```

        "table_data": table_data,
        "table_name": table_name,
        "table_columns": [desc[0] for desc in cursor.description] if 'cursor' in
locals() and table_data else [],
        "selected_db": selected_db,
        "error": error_msg
    }

    return render(request, 'table_view.html', context)

def create_table(request):
    if request.method == "POST":
        pas = request.session.get("mysql_password")
        db = request.session.get("current_db")
        table_name = request.POST.get("tableName")

        try:
            field_count = int(request.POST.get("fieldCount", 0))
        except ValueError:
            return HttpResponse("❌ Invalid number of fields.")

        if not field_count:
            return HttpResponse("❌ Field count missing or invalid.")

        fields = []
        for i in range(1, field_count + 1):
            fname = request.POST.get(f"fieldName{i}")
            ftype = request.POST.get(f"fieldType{i}")
            if not fname or not ftype:
                return HttpResponse(f"❌ Missing name or type for field {i}.")
            fields.append(f"`{fname}` {ftype}")

        q = f"CREATE TABLE `{table_name}` ({', '.join(fields)})"
        try:
            conn = mysql.connector.connect(host="localhost", user="root",
password=pas, database=db)
            cl = conn.cursor()
            cl.execute(q)
            conn.commit()
            conn.close()
            return HttpResponse(f"✅ Table '{table_name}' created successfully.")
        except Exception as e:
            return HttpResponse(f"❌ Error: {e}")

    return render(request, "create_table.html")

def delete_table(request):
    pas = request.session.get("mysql_password")
    db = request.session.get("current_db")
    message = None
    tables = [] # Store list of tables

    try:
        if not pas or not db:
            message = "❌ Not connected to database. Please connect first."
        else:
            # Connect to database
            conn = mysql.connector.connect(
                host="localhost",
                user="root",
                password=pas,
                database=db

```

```

    )
    cursor = conn.cursor()

    # Get list of all tables in the database
    cursor.execute("SHOW TABLES")
    tables = [table[0] for table in cursor.fetchall()]

    if request.method == "POST":
        name = request.POST.get("deltable")

        if not name:
            message = "❌ Please enter a table name."
        elif name not in tables:
            message = f"❌ Table '{name}' does not exist in database '{db}'."
        else:
            # ✅ Get table info before deletion
            cursor.execute(f"SELECT COUNT(*) FROM `{name}`")
            row_count = cursor.fetchone()[0]

            cursor.execute(f"SHOW TABLE STATUS LIKE '{name}'")
            table_info = cursor.fetchone()
            size_info = f"{table_info[6] / 1024:.1f} KB" if table_info and
table_info[6] else "Unknown"

            # ✅ safe table name with backticks
            query = f"DROP TABLE `{name}`"
            cursor.execute(query)
            conn.commit()

            # Remove from tables list after deletion
            tables.remove(name)

            message = f"✅ Table '{name}' deleted successfully. (Rows:
{row_count}, Size: {size_info})"

        cursor.close()
        conn.close()

    except mysql.connector.Error as e:
        message = f"❌ Database Error: {str(e)}"
        if 'cursor' in locals() and cursor:
            cursor.close()
        if 'conn' in locals() and conn:
            conn.close()

    return render(request, "delete_table.html", {
        "message": message,
        "tables": tables,
        "database": db
    })
def edit_table(request):
    table_name = request.session.get("selected_table")
    db_name = request.session.get("selected_db")

    context = {"table": table_name,}
    return render(request, "edit_table.html", context)
def insert(request):
    pas = request.session.get("mysql_password")
    dbname = request.session.get("current_db")
    table = request.session.get("selected_table")

```

```

if request.method == "POST":
    records = request.POST.get("records")

    # STEP 1: User just entered record count
    if records and not request.POST.get("field_0_0"):
        try:
            records = int(records)
        except ValueError:
            return HttpResponse("❌ Invalid record count")

        # fetch columns
        conn = mysql.connector.connect(host="localhost", user="root",
password=pas, database=dbname)
        c1 = conn.cursor()
        c1.execute(f"DESCRIBE {table}")
        columns = [col[0] for col in c1.fetchall()]
        conn.close()

        return render(request, "insert_records.html", {"table": table, "columns":
columns, "records": range(records)})

    # STEP 2: User submitted actual values
    else:
        try:
            conn = mysql.connector.connect(host="localhost", user="root",
password=pas, database=dbname)
            c1 = conn.cursor()

            # fetch columns again
            c1.execute(f"DESCRIBE {table}")
            columns = [col[0] for col in c1.fetchall()]

            records_to_insert = []
            i = 0
            while True:
                row = []
                row_has_data = False
                for col in range(len(columns)):
                    field_value = request.POST.get(f"field_{i}_{col}")
                    if field_value not in [None, ""]:
                        row.append(field_value)
                        row_has_data = True
                    else:
                        row.append(None)
                if not row_has_data:
                    break
                records_to_insert.append(row)
                i += 1

            if records_to_insert:
                placeholders = ", ".join(["%s"] * len(columns))
                query = f"INSERT INTO {table} ({', '.join(columns)}) VALUES
({placeholders})"
                c1.executemany(query, records_to_insert)
                conn.commit()
                msg = f"✅ {len(records_to_insert)} record(s) inserted
successfully into {table}!"
            else:
                msg = "⚠️ No data provided."

            conn.close()
            return HttpResponse(msg)

```

```

        except Error as e:
            return HttpResponse(f"✖ MySQL Error: {e}")

    return render(request, "edit_table.html")

def update_records(request):
    pas = request.session.get("mysql_password")
    dbname = request.session.get("current_db")
    table = request.session.get("selected_table")

    conn = None
    c1 = None
    try:
        conn = mysql.connector.connect(host="localhost", user="root", password=pas,
        database=dbname)
        c1 = conn.cursor(dictionary=True)

        pk_col = None
        c1.execute(f"SHOW KEYS FROM `{table}` WHERE Key_name = 'PRIMARY'")
        primary_key_info = c1.fetchone()
        if primary_key_info:
            pk_col = primary_key_info['Column_name']

        if request.method == "POST":
            row_indices = request.POST.getlist("row_indices")
            if not row_indices:
                return HttpResponse("No rows submitted for update.", status=400)

            conn.autocommit = False
            try:
                for row_index in row_indices:
                    columns = request.POST.getlist(f"row_{row_index}_columns")

                    set_clause_parts = []
                    update_params = []
                    for col in columns:
                        new_value = request.POST.get(f"row_{row_index}_{col}")
                        processed_value = None if new_value in ["", "NULL"] else
new_value

                        set_clause_parts.append(f"`{col}` = %s")
                        update_params.append(processed_value)

                    set_clause = ", ".join(set_clause_parts)

                    where_params = []
                    if pk_col:
                        original_pk_value =
request.POST.get(f"original_row_{row_index}_pk")
                        where_clause = f"`{pk_col}` = %s"
                        where_params.append(original_pk_value)
                    else:
                        where_clause_parts = []
                        for col in columns:
                            original_value =
request.POST.get(f"original_row_{row_index}_{col}")
                            processed_original_value = None if original_value in ["",
"NULL"] else original_value
                            where_clause_parts.append(f"`{col}` <=> %s")
                            where_params.append(processed_original_value)
                        where_clause = " AND ".join(where_clause_parts)

```



```

        update_query = f"UPDATE `{table}` SET {set_clause} WHERE
{where_clause}"
        cl.execute(update_query, update_params + where_params)

        conn.commit()
        return HttpResponse("✅ Records updated successfully!")
    except Exception as e:
        conn.rollback()
        return HttpResponse(f"❌ Transaction failed. Error: {e}", status=500)
    else:
        cl.execute(f"SELECT * FROM `{table}` LIMIT 100")
        table_data = cl.fetchall()
        columns = [desc[0] for desc in cl.description]

        return render(request, "update_records.html", {"table_name":
table,"table_columns": columns,"table_data": table_data,"selected_db":
dbname,"pk_col": pk_col})

    except mysql.connector.Error as e:
        return HttpResponse(f"❌ MySQL Error: {e}", status=500)
    except Exception as e:
        return HttpResponse(f"❌ An unexpected server error occurred: {e}",
status=500)
    finally:
        if cl:
            cl.close()
        if conn:
            conn.close()

def delete_record(request):
    pas = request.session.get("mysql_password")
    dbname = request.session.get("current_db")
    table = request.session.get("selected_table")

    conn = None
    cl = None
    try:
        conn = mysql.connector.connect(
            host="localhost",
            user="root",
            password=pas,
            database=dbname
        )
        cl = conn.cursor(dictionary=True)

        # Identify primary key column
        pk_col = None
        cl.execute(f"SHOW KEYS FROM `{table}` WHERE Key_name = 'PRIMARY'")
        primary_key_info = cl.fetchone()
        if primary_key_info:
            pk_col = primary_key_info['Column_name']

        if request.method == "POST":
            conn.autocommit = False
            try:
                if pk_col:
                    # Delete by primary key
                    pk_value = request.POST.get("pk_value")
                    delete_query = f"DELETE FROM `{table}` WHERE `{pk_col}` = %s"
                    cl.execute(delete_query, (pk_value,))
                else:
                    # No PK → delete using all column values

```

```

        where_parts = []
        where_params = []
        for key, value in request.POST.items():
            if key.startswith("col_"):
                col = key.replace("col_", "")
                processed_value = None if value in ["", "NULL"] else value
                where_parts.append(f"`{col}` <=> %s")
                where_params.append(processed_value)

        where_clause = " AND ".join(where_parts)
        delete_query = f"DELETE FROM `{table}` WHERE {where_clause} LIMIT
1"

        c1.execute(delete_query, where_params)

        conn.commit()
        return redirect("/update_records/") # back to update page
    except Exception as e:
        conn.rollback()
        return HttpResponse(f"❌ Delete failed. Error: {e}", status=500)

    return HttpResponse("Invalid request method", status=405)

except mysql.connector.Error as e:
    return HttpResponse(f"❌ MySQL Error: {e}", status=500)
except Exception as e:
    return HttpResponse(f"❌ Unexpected error: {e}", status=500)
finally:
    if c1:
        c1.close()
    if conn:
        conn.close()
})

```

5.4 urls.py — URL Routing

"""

URL configuration for mysql_manager project.

The `urlpatterns` list routes URLs to views. For more information please see:

<https://docs.djangoproject.com/en/5.2/topics/http/urls/>

Examples:

Function views

1. Add an import: `from my_app import views`
2. Add a URL to urlpatterns: `path('', views.home, name='home')`

Class-based views

1. Add an import: `from other_app.views import Home`
2. Add a URL to urlpatterns: `path('', Home.as_view(), name='home')`

Including another URLconf

1. Import the include() function: `from django.urls import include, path`
2. Add a URL to urlpatterns: `path('blog/', include('blog.urls'))`

"""

from django.contrib import admin

from django.urls import path

from mysql import views

```

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.login),
    path('home/', views.home),
    path('explore/', views.explore),
    path('get_dbs/', views.get_dbs),

```

```
path('create_dbs/',views.create_dbs),
path('delete_dbs/',views.delete_dbs),
path('work/',views.work),
path('select_db/',views.select_db),
path('show_tables/',views.show_tables),
path('view_table/', views.view_table),
path('create_table/',views.create_table),
path('delete_table/',views.delete_table),
path('edit_table/',views.edit_table),
path('insert/',views.insert),
path('insert_records/',views.insert),
path('update_records/',views.update_records),
path('delete_record/',views.delete_record),
]
```

5.5 login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>MySQL Manager | Secure Login</title>
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
  <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600&display=swap"
rel="stylesheet">

  <style>
    /* — Reset & Base — */
    *, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }

    * { -webkit-font-smoothing: antialiased; -moz-osx-font-smoothing: grayscale;
cursor: none !important; }

    html, body {
      height: 100%;
      background: linear-gradient(135deg, #0f172a 0%, #1e293b 100%);
      font-family: 'Inter', sans-serif;
      color: #e5e7eb;
      overflow: hidden; /* login page, single viewport */
    }

    /* — Custom Cursor — */
    .custom-cursor {
      position: fixed; width: 16px; height: 16px;
      border: 1.5px solid rgba(59,130,246,0.85);
      border-radius: 50%; pointer-events: none; z-index: 9999;
      mix-blend-mode: screen;
      transition: width .08s ease, height .08s ease, background .08s ease, border-
color .08s ease;
      will-change: transform; backface-visibility: hidden; transform:
translateZ(0);
    }
    .cursor-dot {
      position: fixed; width: 4px; height: 4px;
      background: #3b82f6; border-radius: 50%;
      pointer-events: none; z-index: 9999; mix-blend-mode: screen;
      will-change: transform; backface-visibility: hidden; transform:
translateZ(0);
```

```
}

/* — Background Grid — */
.database-grid {
  position: fixed; inset: 0;
  background-image:
    linear-gradient(rgba(59,130,246,0.03) 1px, transparent 1px),
    linear-gradient(90deg, rgba(59,130,246,0.03) 1px, transparent 1px);
  background-size: 32px 32px;
  background-position: center;
  z-index: 0; pointer-events: none;
}

/* — Static Data Nodes — */
.data-node {
  position: fixed; width: 5px; height: 5px;
  background: rgba(59,130,246,0.5); border-radius: 50%;
  z-index: 2; box-shadow: 0 0 8px rgba(59,130,246,0.3);
}

/* — Centred Layout — */
.page-center {
  position: relative; z-index: 10;
  min-height: 100vh;
  display: flex; flex-direction: column;
  align-items: center; justify-content: center;
  padding: 24px;
}

/* — Login Card (mirrors card-hover) — */
.login-card {
  width: 380px;
  background: rgba(30,41,59,0.9);
  backdrop-filter: blur(8px);
  border: 1px solid rgba(59,130,246,0.12);
  border-radius: 16px;
  padding: 40px 36px 36px;
  transition: border-color .2s cubic-bezier(.4,0,.2,1), box-shadow .2s cubic-
bezier(.4,0,.2,1);
  animation: slideUp .35s cubic-bezier(.4,0,.2,1) both;
}
.login-card:hover {
  border-color: rgba(59,130,246,0.25);
  box-shadow: 0 8px 20px rgba(59,130,246,0.15);
}

/* shake on error */
.login-card.shake {
  animation: shake .45s cubic-bezier(.4,0,.2,1);
}

/* — Keyframes — */
@keyframes slideUp {
  from { opacity: 0; transform: translateY(14px); }
  to   { opacity: 1; transform: translateY(0); }
}
@keyframes shake {
  0%,100% { transform: translateX(0); }
  20%     { transform: translateX(-6px); }
  40%     { transform: translateX(6px); }
  60%     { transform: translateX(-4px); }
  80%     { transform: translateX(4px); }
}
```

```
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(4px); }
  to   { opacity: 1; transform: translateY(0); }
}

/* — Card Header — */
.card-header { text-align: center; margin-bottom: 32px; }

.icon-wrap {
  width: 56px; height: 56px; margin: 0 auto 18px;
  background: linear-gradient(135deg, rgba(59,130,246,0.2),
    rgba(37,99,235,0.1));
  border: 1px solid rgba(59,130,246,0.2);
  border-radius: 14px;
  display: flex; align-items: center; justify-content: center;
}
.icon-wrap i { font-size: 1.3rem; color: #60a5fa; }

.card-header h1 {
  font-size: 1.25rem; font-weight: 600; color: #fff;
  margin-bottom: 8px;
}

.card-header p {
  font-size: 0.78rem; color: #6b7280; line-height: 1.6;
}

/* server-status pill */
.status-pill {
  display: inline-flex; align-items: center; gap: 6px;
  margin-top: 12px;
  background: rgba(16,185,129,0.08);
  border: 1px solid rgba(16,185,129,0.2);
  border-radius: 20px;
  padding: 4px 10px;
}
.status-pill .ping-wrap {
  position: relative; width: 7px; height: 7px;
}
.status-pill .ping-wrap .ping-outer {
  position: absolute; inset: 0;
  background: #10b981; border-radius: 50%;
  animation: ping 2s cubic-bezier(0,0,.2,1) infinite;
  opacity: .5;
}
.status-pill .ping-wrap .ping-inner {
  position: relative; z-index: 1;
  width: 7px; height: 7px;
  background: #10b981; border-radius: 50%;
}
@keyframes ping {
  to { transform: scale(2.2); opacity: 0; }
}
.status-pill span { font-size: 0.68rem; color: #6ee7b7; font-weight: 500; }

/* — Label — */
.field-label {
  display: flex; align-items: center; gap: 6px;
  font-size: 0.78rem; font-weight: 600;
  color: #9ca3af; text-transform: uppercase; letter-spacing: .4px;
  margin-bottom: 10px;
}
.field-label i { color: #60a5fa; font-size: 0.72rem; }
```

```
/* — Input (mirrors input-premium) — */
.input-premium {
  width: 100%;
  background: rgba(30,41,59,0.7);
  border: 1px solid rgba(59,130,246,0.2);
  border-radius: 8px;
  padding: 13px 42px 13px 16px;
  font-size: 0.85rem; color: #fff;
  font-family: inherit;
  transition: all .15s cubic-bezier(.4,0,.2,1);
}
.input-premium::placeholder { color: #4b5563; }
.input-premium:focus {
  outline: none;
  border-color: rgba(59,130,246,0.7);
  box-shadow: 0 0 0 2px rgba(59,130,246,0.15);
  background: rgba(30,41,59,0.9);
}

/* toggle eye button */
.toggle-pw {
  position: absolute; right: 14px; top: 50%; transform: translateY(-50%);
  background: none; border: none;
  color: #4b5563; cursor: pointer !important;
  font-size: 0.8rem;
  transition: color .15s ease;
  padding: 4px;
}
.toggle-pw:hover { color: #60a5fa; }

.input-wrap { position: relative; }

/* helper row under input */
.input-helper {
  display: flex; justify-content: space-between; align-items: center;
  margin-top: 8px;
}
.input-helper span {
  font-size: 0.65rem; color: #4b5563;
  display: flex; align-items: center; gap: 4px;
}
.input-helper span i { font-size: 0.6rem; }
.input-helper .helper-accent { color: #60a5fa; }

/* — Submit Button (mirrors btn btn-primary) — */
.btn-connect {
  width: 100%;
  background: linear-gradient(135deg, #3b82f6, #2563eb);
  border: 1px solid rgba(59,130,246,0.3);
  border-radius: 8px;
  padding: 13px 20px;
  color: #fff; font-size: 0.85rem; font-weight: 600;
  font-family: inherit;
  cursor: pointer !important;
  display: inline-flex; align-items: center; justify-content: center; gap: 8px;
  transition: all .15s cubic-bezier(.4,0,.2,1);
  margin-top: 24px;
}
.btn-connect:hover:not(:disabled) {
  transform: translateY(-1px);
  box-shadow: 0 4px 12px rgba(59,130,246,0.3);
}
```

```
.btn-connect:active:not(:disabled) { transform: translateY(0); }
.btn-connect:disabled { opacity: .6; }
.btn-connect i { font-size: 0.8rem; }

/* — Alerts — */
.alert-box {
  border-radius: 8px; padding: 12px 16px;
  display: flex; align-items: center; gap: 10px;
  font-size: 0.78rem; line-height: 1.5;
  animation: fadeIn .25s ease both;
  margin-top: 16px;
}
.alert-box .alert-icon {
  flex-shrink: 0; width: 24px; height: 24px;
  border-radius: 50%;
  display: flex; align-items: center; justify-content: center;
  font-size: 0.7rem;
}

/* error variant */
.alert-error {
  background: rgba(239,68,68,0.1);
  border: 1px solid rgba(239,68,68,0.2);
  color: #fca5a5;
}
.alert-error .alert-icon { background: rgba(239,68,68,0.15); color: #f87171; }

/* validation (amber) variant */
.alert-validation {
  background: rgba(245,158,11,0.1);
  border: 1px solid rgba(245,158,11,0.2);
  color: #fcd34d;
}
.alert-validation .alert-icon { background: rgba(245,158,11,0.15); color: #fbbf24; }

/* — Bottom-right status indicator — */
.status-corner {
  position: fixed; bottom: 20px; right: 20px; z-index: 20;
  background: rgba(15,23,42,0.85);
  backdrop-filter: blur(6px);
  border: 1px solid rgba(59,130,246,0.12);
  border-radius: 8px;
  padding: 6px 12px;
  display: flex; align-items: center; gap: 8px;
}
.status-corner .sc-ping { position: relative; width: 7px; height: 7px; }
.status-corner .sc-ping .sc-outer {
  position: absolute; inset: 0;
  background: #10b981; border-radius: 50%;
  animation: ping 2s cubic-bezier(0,0,.2,1) infinite; opacity: .4;
}
.status-corner .sc-ping .sc-inner {
  position: relative; z-index: 1;
  width: 7px; height: 7px;
  background: #10b981; border-radius: 50%;
}
.status-corner .sc-text { font-size: 0.68rem; }
.status-corner .sc-text .sc-host { color: #9ca3af; }
.status-corner .sc-text .sc-live { color: #34d399; font-weight: 500; margin-left:
5px; }

/* — Utility — */
```

```
        .text-xs-sm { font-size: 0.7rem; }
    </style>
</head>
<body>
    <!-- Cursor -->
    <div class="custom-cursor"></div>
    <div class="cursor-dot"></div>

    <!-- Background -->
    <div class="database-grid"></div>
    <div class="data-node" style="top:12%;left:8%;"></div>
    <div class="data-node" style="top:75%;right:12%;"></div>
    <div class="data-node" style="bottom:30%;left:18%;"></div>
    <div class="data-node" style="top:45%;right:22%;"></div>
    <div class="data-node" style="top:60%;left:55%;"></div>

    <!-- Page centre -->
    <div class="page-center">

        <form method="post" id="loginForm" novalidate>
            {% csrf_token %}
            <div class="login-card" id="loginCard">

                <!-- Header -->
                <div class="card-header">
                    <div class="icon-wrap">
                        <i class="fas fa-lock"></i>
                    </div>
                    <h1>MySQL Database Login</h1>
                    <p>Enter your system password to establish<br>a secure database
connection</p>

                    <div class="status-pill">
                        <div class="ping-wrap">
                            <div class="ping-outer"></div>
                            <div class="ping-inner"></div>
                        </div>
                        <span>MySQL Server Running</span>
                    </div>
                </div>

                <!-- Password field -->
                <div class="field-group">
                    <label class="field-label">
                        <i class="fas fa-key"></i> System Password
                    </label>

                    <div class="input-wrap">
                        <input type="password"
                            name="p"
                            id="passwordInput"
                            autocomplete="current-password"
                            placeholder="Enter your password"
                            class="input-premium">

                        <button type="button" class="toggle-pw" id="toggleBtn"
onclick="togglePassword()" aria-label="Toggle password visibility">
                            <i class="fas fa-eye" id="toggleIcon"></i>
                        </button>
                    </div>

                    <div class="input-helper">
                        <span><i class="fas fa-shield-alt"></i> Secure connection</span>
                    </div>
                </div>
            </div>
        </form>
    </div>
</body>
</html>
```



```
Encrypted</span>                <span class="helper-accent"><i class="fas fa-lock"></i>
                                </div>
                                </div>

                                <!-- Submit -->
                                <button type="submit" class="btn-connect" id="submitBtn">
                                    <i class="fas fa-plug"></i> Connect to Database
                                </button>

                                <!-- Django error messages (server-side) -->
                                {% if field_error %}
                                <div class="alert-box alert-error">
                                    <div class="alert-icon"><i class="fas fa-exclamation-
triangle"></i></div>
                                    <span>{{ field_error }}</span>
                                </div>
                                {% endif %}

                                {% if error %}
                                <div class="alert-box alert-error">
                                    <div class="alert-icon"><i class="fas fa-times-circle"></i></div>
                                    <span>{{ error }}</span>
                                </div>
                                {% endif %}

                                <!-- JS validation target -->
                                <div id="validationTarget"></div>

                                </div><!-- /login-card -->
                            </form>
                        </div><!-- /page-center -->

                        <!-- Bottom-right status -->
                        <div class="status-corner">
                            <div class="sc-ping">
                                <div class="sc-outer"></div>
                                <div class="sc-inner"></div>
                            </div>
                            <div class="sc-text">
                                <span class="sc-host">MySQL:3306</span>
                                <span class="sc-live">Listening</span>
                            </div>
                        </div>
                    </div>

                    <!-- — Scripts — -->
                    <script>
                    (function(){
                        /* ——— Cursor ——— */
                        const cursor    = document.querySelector('.custom-cursor');
                        const cursorDot  = document.querySelector('.cursor-dot');
                        let mouseX=0, mouseY=0, cursorX=0, cursorY=0;

                        document.addEventListener('mousemove', e => {
                            mouseX = e.clientX; mouseY = e.clientY;
                            cursorDot.style.transform = `translate(${mouseX}px,${mouseY}px)`;
                            cursorX += (mouseX - cursorX) * 0.5;
                            cursorY += (mouseY - cursorY) * 0.5;
                            cursor.style.transform = `translate(${cursorX}px,${cursorY}px)`;
                        }, { passive: true });

                        function setupHoverEffects(){
                            const els = [
```

```
        ...document.querySelectorAll('a, button, .login-card, input')
    ];
    els.forEach(el => {
        el.addEventListener('mouseenter', () => {
            cursor.style.width = '28px';
            cursor.style.height = '28px';
            cursor.style.borderColor = 'rgba(96,165,250,0.9)';
            cursor.style.background = 'rgba(59,130,246,0.08)';
        }, { passive: true });
        el.addEventListener('mouseleave', () => {
            cursor.style.width = '16px';
            cursor.style.height = '16px';
            cursor.style.borderColor = 'rgba(59,130,246,0.85)';
            cursor.style.background = 'transparent';
        }, { passive: true });
    });
    document.addEventListener('mousedown', () => { cursor.style.width='12px';
    cursor.style.height='12px'; }, {passive:true});
    document.addEventListener('mouseup', () => { cursor.style.width='16px';
    cursor.style.height='16px'; }, {passive:true});

    window.addEventListener('load', () => {
        cursorX = innerWidth/2; cursorY = innerHeight/2;
        mouseX = cursorX; mouseY = cursorY;
        cursor.style.transform = `translate(${cursorX}px,${cursorY}px)`;
        cursorDot.style.transform = `translate(${cursorX}px,${cursorY}px)`;
        setupHoverEffects();
        document.getElementById('passwordInput').focus();
    });

    /* ----- Password toggle ----- */
    window.togglePassword = function(){
        const input = document.getElementById('passwordInput');
        const icon = document.getElementById('toggleIcon');
        if(input.type === 'password'){
            input.type = 'text';
            icon.classList.replace('fa-eye', 'fa-eye-slash');
        } else {
            input.type = 'password';
            icon.classList.replace('fa-eye-slash', 'fa-eye');
        }
    };

    /* ----- Form logic ----- */
    const form = document.getElementById('loginForm');
    const card = document.getElementById('loginCard');
    const submitBtn = document.getElementById('submitBtn');
    const pwInput = document.getElementById('passwordInput');
    const valTarget = document.getElementById('validationTarget');

    function clearValidation(){
        valTarget.innerHTML = '';
    }

    function triggerShake(){
        card.classList.remove('shake');
        void card.offsetWidth; // reflow
        card.classList.add('shake');
        setTimeout(() => card.classList.remove('shake'), 500);
    }

    form.addEventListener('submit', function(e){
```

```
        if(!pwInput.value.trim()){
            e.preventDefault();
            clearValidation();
            valTarget.innerHTML = `
                <div class="alert-box alert-validation">
                    <div class="alert-icon"><i class="fas fa-exclamation-
circle"></i></div>
                        <span>Please enter your system password to continue.</span>
                    </div>`;
            triggerShake();
        } else {
            // valid — show spinner, let form submit
            submitBtn.innerHTML = '<i class="fas fa-spinner fa-spin"></i>
Connecting...';
            submitBtn.disabled = true;
        }
    });

    // clear validation on input
    pwInput.addEventListener('input', () => {
        clearValidation();
        if(submitBtn.disabled){
            submitBtn.innerHTML = '<i class="fas fa-plug"></i> Connect to Database';
            submitBtn.disabled = false;
        }
    });

    // restore button on bfcache
    window.addEventListener('pageshow', ev => {
        if(ev.persisted){
            submitBtn.innerHTML = '<i class="fas fa-plug"></i> Connect to Database';
            submitBtn.disabled = false;
        }
    });
})();
</script>
</body>
</html>
```

5.6 home.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8" />
        <meta name="viewport" content="width=device-width, initial-scale=1.0" />
        <title>MySQL Manager | Dashboard</title>
        <script src="https://cdn.tailwindcss.com"></script>
        <link
            rel="stylesheet"
            href="https://cdn.jsdelivr.net/npm/font-awesome@6.4.0/css/all.min.css"
        />
        <link
            href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600&display=s
wap"
            rel="stylesheet"
        />
    <style>
```

```
* {
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  cursor: none !important;
}

html {
  scroll-behavior: smooth;
}

body {
  background: linear-gradient(135deg, #0f172a 0%, #1e293b 100%);
  min-height: 100vh;
  position: relative;
  overflow-x: hidden;
  font-family: "Inter", sans-serif;
}

/* Optimized Custom Cursor */
.custom-cursor {
  position: fixed;
  width: 18px;
  height: 18px;
  border: 2px solid rgba(59, 130, 246, 0.85);
  border-radius: 50%;
  pointer-events: none;
  z-index: 9999;
  mix-blend-mode: screen;
  transition: width 0.15s ease, height 0.15s ease, background 0.15s ease, border-
color 0.15s ease;
  will-change: transform;
  backface-visibility: hidden;
  transform: translateZ(0);
}

.cursor-dot {
  position: fixed;
  width: 5px;
  height: 5px;
  background: #3b82f6;
  border-radius: 50%;
  pointer-events: none;
  z-index: 9999;
  mix-blend-mode: screen;
  will-change: transform;
  backface-visibility: hidden;
  transform: translateZ(0);
}

/* Database Grid - Optimized */
.database-grid {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-image: linear-gradient(
    rgba(59, 130, 246, 0.03) 1px,
    transparent 1px
  ),
    linear-gradient(90deg, rgba(59, 130, 246, 0.03) 1px, transparent 1px);
  background-size: 35px 35px;
  background-position: center center;
```

```
    z-index: 0;
    pointer-events: none;
}

/* Static data nodes */
.data-node {
    position: fixed;
    width: 6px;
    height: 6px;
    background: rgba(59, 130, 246, 0.5);
    border-radius: 50%;
    z-index: 2;
    box-shadow: 0 0 10px rgba(59, 130, 246, 0.3);
}

.nav-glass {
    background: rgba(15, 23, 42, 0.92);
    backdrop-filter: blur(8px);
    border-bottom: 1px solid rgba(59, 130, 246, 0.08);
    z-index: 50;
}

.card-hover {
    background: rgba(30, 41, 59, 0.9);
    backdrop-filter: blur(8px);
    border: 1px solid rgba(59, 130, 246, 0.12);
    transition: all 0.25s cubic-bezier(0.4, 0, 0.2, 1);
}

.card-hover:hover {
    border-color: rgba(59, 130, 246, 0.25);
    transform: translateY(-1px);
    box-shadow: 0 12px 25px rgba(59, 130, 246, 0.15);
}

.hero-gradient {
    background: linear-gradient(
        90deg,
        #60a5fa 0%,
        #3b82f6 50%,
        #2563eb 100%
    );
    -webkit-background-clip: text;
    background-clip: text;
    -webkit-text-fill-color: transparent;
}

.feature-icon {
    width: 36px;
    height: 36px;
    background: linear-gradient(
        135deg,
        rgba(59, 130, 246, 0.15),
        rgba(37, 99, 235, 0.1)
    );
    border: 1px solid rgba(59, 130, 246, 0.25);
    backdrop-filter: blur(5px);
    transition: all 0.2s ease;
}

/* Text sizes */
.text-xs-sm {
    font-size: 0.75rem;
}
```

```
    }

    .text-sm-sm {
        font-size: 0.8125rem;
    }
</style>
</head>

<body class="font-inter text-gray-200">
    <!-- Custom Cursor -->
    <div class="custom-cursor"></div>
    <div class="cursor-dot"></div>

    <!-- Static Database Grid -->
    <div class="database-grid"></div>

    <!-- Static Data Nodes -->
    <div class="data-node" style="top: 15%; left: 10%;"></div>
    <div class="data-node" style="top: 70%; right: 15%;"></div>
    <div class="data-node" style="bottom: 40%; left: 20%;"></div>
    <div class="data-node" style="top: 50%; right: 25%;"></div>
    <div class="data-node" style="top: 30%; left: 65%;"></div>
    <div class="data-node" style="bottom: 60%; right: 35%;"></div>
    <div class="data-node" style="top: 85%; left: 45%;"></div>
    <div class="data-node" style="bottom: 20%; right: 55%;"></div>

    <!-- Navigation -->
    <nav class="nav-glass sticky top-0">
        <div class="container mx-auto px-5 py-2.5">
            <div class="flex justify-between items-center">
                <div class="flex items-center space-x-2">
                    <div class="w-7 h-7 bg-gradient-to-br from-blue-500/25 to-blue-600/10
rounded-lg flex items-center justify-center border border-blue-500/20">
                        <i class="fas fa-database text-blue-400 text-xs"></i>
                    </div>
                    <span class="text-base font-semibold text-white">MySQL Manager</span>
                </div>
                <span
                    class="text-xs px-2 py-0.5 bg-blue-500/20 text-blue-300 rounded-full"
                    >v2.0</span>
            </div>

            <div class="flex items-center space-x-4">
                <a
                    href="#features"
                    class="text-xs-sm text-gray-300 hover:text-blue-400 transition-colors
duration-150"
                >
                    Features
                </a>
                <div
                    class="px-2.5 py-1 rounded-full bg-emerald-500/10 text-emerald-400 text-xs-
sm font-medium border border-emerald-400/25"
                >
                    <span
                        class="relative inline-flex h-1.5 w-1.5 rounded-full bg-emerald-400"
                    ></span>
                    </span>
                    Connected
                </div>
            </div>
        </div>
    </nav>
</body>
```

```
</div>
</nav>

<!-- Main Content -->
<main class="container mx-auto px-5 py-8 relative z-10">
  <!-- Hero Section -->
  <div class="text-center max-w-3xl mx-auto">
    <div
      class="inline-flex items-center justify-center w-16 h-16 bg-gradient-to-br
from-blue-500/20 to-blue-600/10 rounded-2xl mb-4 border border-blue-500/20"
    >
      <i class="fas fa-server text-2xl text-blue-400"></i>
    </div>

    <h1
      class="text-3xl font-bold mb-3 hero-gradient"
    >
      Modern MySQL Management
    </h1>

    <p class="text-gray-300 mb-6 text-sm-sm leading-relaxed max-w-2xl mx-auto">
      A sleek, intuitive interface for managing MySQL databases. Explore,
      edit, and optimize your databases with powerful visual tools.
    </p>
  </div>

  <!-- Action Cards -->
  <div
    class="grid grid-cols-1 md:grid-cols-2 gap-6 max-w-4xl mx-auto mt-10"
  >
    <!-- Card 1 - Database Explorer -->
    <a
      href="http://127.0.0.1:8000/explore/"
      class="card-hover rounded-xl p-5 hover:bg-blue-900/10"
    >
      <div class="flex items-start space-x-3">
        <div
          class="feature-icon rounded-lg flex items-center justify-center flex-
shrink-0"
        >
          <i class="fas fa-search text-blue-400 text-sm"></i>
        </div>
        <div class="flex-1">
          <div class="flex items-center justify-between mb-2">
            <h3 class="text-base font-semibold text-white">
              Database Explorer
            </h3>
            <i
              class="fas fa-arrow-right text-blue-400 text-xs"
            ></i>
          </div>
          <p class="text-gray-400 text-xs-sm mb-4 leading-relaxed">
            Browse databases, tables, and records with an intuitive visual
            interface. Real-time preview and schema visualization.
          </p>
          <div class="space-y-2">
            <div
              class="flex items-center text-gray-300 text-xs-sm"
            >
              <div
                class="w-1.5 h-1.5 bg-blue-400 rounded-full mr-2"
              ></div>
              <span>Visual table explorer</span>
            </div>
          </div>
        </div>
      </div>
    </a>
  </div>
</main>
```

```
</div>
<div
  class="flex items-center text-gray-300 text-xs-sm"
>
  <div
    class="w-1.5 h-1.5 bg-blue-400 rounded-full mr-2"
  ></div>
  <span>Real-time data preview</span>
</div>
<div
  class="flex items-center text-gray-300 text-xs-sm"
>
  <div
    class="w-1.5 h-1.5 bg-blue-400 rounded-full mr-2"
  ></div>
  <span>Schema visualization</span>
</div>
</div>
</div>
</div>
</a>

<!-- Card 2 - Query Workshop -->
<a
  href="http://127.0.0.1:8000/work/"
  class="card-hover rounded-xl p-5 hover:bg-green-900/10"
>
  <div class="flex items-start space-x-3">
    <div
      class="feature-icon rounded-lg flex items-center justify-center flex-
shrink-0"
    >
      <i class="fas fa-edit text-green-400 text-sm"></i>
    </div>
    <div class="flex-1">
      <div class="flex items-center justify-between mb-2">
        <h3 class="text-base font-semibold text-white">Query Workshop</h3>
        <i
          class="fas fa-arrow-right text-green-400 text-xs"
        ></i>
      </div>
      <p class="text-gray-400 text-xs-sm mb-4 leading-relaxed">
        Create, edit, and execute SQL queries with syntax highlighting
        and auto-complete. Save and manage query history.
      </p>
      <div class="space-y-2">
        <div
          class="flex items-center text-gray-300 text-xs-sm"
        >
          <div
            class="w-1.5 h-1.5 bg-green-400 rounded-full mr-2"
          ></div>
          <span>Visual query builder</span>
        </div>
        <div
          class="flex items-center text-gray-300 text-xs-sm"
        >
          <div
            class="w-1.5 h-1.5 bg-green-400 rounded-full mr-2"
          ></div>
          <span>SQL auto-completion</span>
        </div>
      </div>
    </div>
  </div>
</a>
```



```

        class="flex items-center text-gray-300 text-xs-sm"
      >
      <div
        class="w-1.5 h-1.5 bg-green-400 rounded-full mr-2"
      ></div>
      <span>Query history & favorites</span>
    </div>
  </div>
</div>
</a>
</div>

<!-- Features Section -->
<div class="max-w-4xl mx-auto mt-12" id="features">
  <div class="text-center mb-8">
    <h2 class="text-xl font-bold text-white mb-2">Powerful Features</h2>
    <p class="text-gray-400 text-xs-sm">
      Everything you need for efficient database management
    </p>
  </div>

  <div class="grid grid-cols-1 md:grid-cols-3 gap-4">
    <div
      class="card-hover rounded-lg p-5 transition-all duration-200 hover:scale-
[1.02]"
    >
      <div
        class="w-10 h-10 bg-gradient-to-br from-blue-500/20 to-blue-600/10 rounded-
xl flex items-center justify-center mb-3 border border-blue-500/20"
      >
        <i class="fas fa-chart-line text-blue-400 text-sm"></i>
      </div>
      <h4 class="text-sm font-semibold text-white mb-2">
        Performance Analytics
      </h4>
      <p class="text-gray-400 text-xs-sm leading-relaxed">
        Monitor query performance and optimize database operations with
        detailed insights and visual metrics.
      </p>
    </div>

    <div
      class="card-hover rounded-lg p-5 transition-all duration-200 hover:scale-
[1.02]"
    >
      <div
        class="w-10 h-10 bg-gradient-to-br from-green-500/20 to-green-600/10
rounded-xl flex items-center justify-center mb-3 border border-green-500/20"
      >
        <i class="fas fa-shield-alt text-green-400 text-sm"></i>
      </div>
      <h4 class="text-sm font-semibold text-white mb-2">
        Security Tools
      </h4>
      <p class="text-gray-400 text-xs-sm leading-relaxed">
        Manage user permissions and audit database activities securely
        with role-based access control.
      </p>
    </div>

    <div

```

```

        class="card-hover rounded-lg p-5 transition-all duration-200 hover:scale-
[1.02]"
    >
        <div
            class="w-10 h-10 bg-gradient-to-br from-purple-500/20 to-purple-600/10
rounded-xl flex items-center justify-center mb-3 border border-purple-500/20"
        >
            <i class="fas fa-download text-purple-400 text-sm"></i>
        </div>
        <h4 class="text-sm font-semibold text-white mb-2">Import/Export</h4>
        <p class="text-gray-400 text-xs-sm leading-relaxed">
            Easily import and export data in multiple formats (CSV, JSON, SQL)
            with batch processing support.
        </p>
    </div>
</div>
</div>

<!-- Quick Stats -->
<div class="max-w-4xl mx-auto mt-12">
    <div class="grid grid-cols-2 md:grid-cols-4 gap-4">
        <div
            class="bg-gray-900/40 rounded-lg p-4 text-center border border-gray-800/50
transition-all duration-200 hover:border-blue-500/30 hover:scale-[1.03]"
        >
            <div class="text-blue-400 text-lg font-bold mb-1">MySQL 8.0</div>
            <div class="text-gray-300 text-xs-sm font-medium">Database Engine</div>
        </div>
        <div
            class="bg-gray-900/40 rounded-lg p-4 text-center border border-gray-800/50
transition-all duration-200 hover:border-green-500/30 hover:scale-[1.03]"
        >
            <div class="text-green-400 text-lg font-bold mb-1">Real-time</div>
            <div class="text-gray-300 text-xs-sm font-medium">Live Updates</div>
        </div>
        <div
            class="bg-gray-900/40 rounded-lg p-4 text-center border border-gray-800/50
transition-all duration-200 hover:border-purple-500/30 hover:scale-[1.03]"
        >
            <div class="text-purple-400 text-lg font-bold mb-1">Secure</div>
            <div class="text-gray-300 text-xs-sm font-medium">Encrypted</div>
        </div>
        <div
            class="bg-gray-900/40 rounded-lg p-4 text-center border border-gray-800/50
transition-all duration-200 hover:border-amber-500/30 hover:scale-[1.03]"
        >
            <div class="text-amber-400 text-lg font-bold mb-1">Web-based</div>
            <div class="text-gray-300 text-xs-sm font-medium">Access Anywhere</div>
        </div>
    </div>
</div>
</main>

<!-- Footer -->
<footer class="mt-16 border-t border-gray-800/50 py-8 relative z-10">
    <div class="container mx-auto px-5">
        <div class="flex flex-col md:flex-row justify-between items-center">
            <div class="flex items-center space-x-2.5 mb-4 md:mb-0">
                <div
                    class="w-7 h-7 bg-gradient-to-br from-blue-500/20 to-blue-600/10 rounded
flex items-center justify-center border border-blue-500/20"
                >
                    <i class="fas fa-database text-blue-400 text-xs"></i>

```

```
    </div>
    <div>
      <span class="text-sm font-medium text-white">MySQL Manager</span>
      <p class="text-gray-500 text-xs-sm mt-0.5">
        Modern Database Interface
      </p>
    </div>
  </div>

  <div class="flex items-center space-x-2 text-gray-400 text-xs-sm mb-4 md:mb-0">
    <i class="fas fa-heart text-red-400 animate-pulse"></i>
    <span>Made for database Management</span>
  </div>

  <div>
    <div
      class="flex items-center text-xs-sm text-gray-400 bg-gray-800/40 px-3 py-
1.5 rounded-full"
    >
      <div
        class="w-1.5 h-1.5 bg-green-400 rounded-full mr-2 animate-pulse"
      ></div>
      <span>Connected to MySQL Server</span>
      <span class="text-green-400 ml-2 font-medium">✓</span>
    </div>
  </div>
</div>
</footer>

<!-- Working JavaScript -->
<script>
  // Get cursor elements
  const cursor = document.querySelector(".custom-cursor");
  const cursorDot = document.querySelector(".cursor-dot");

  // Mouse position tracking
  let mouseX = 0;
  let mouseY = 0;

  // Cursor position for smooth interpolation
  let cursorX = 0;
  let cursorY = 0;
  let dotX = 0;
  let dotY = 0;

  // Animation control
  let isAnimating = false;
  let animationId = null;

  // Update mouse position
  document.addEventListener("mousemove", (e) => {
    mouseX = e.clientX;
    mouseY = e.clientY;

    // Start animation if not already running
    if (!isAnimating) {
      isAnimating = true;
      animateCursor();
    }
  });

  // Smooth cursor animation
```

```
function animateCursor() {
  // Smooth interpolation for outer cursor (slower)
  cursorX += (mouseX - cursorX) * 0.15;
  cursorY += (mouseY - cursorY) * 0.15;

  // Faster interpolation for dot (follows more closely)
  dotX += (mouseX - dotX) * 0.3;
  dotY += (mouseY - dotY) * 0.3;

  // Apply transforms
  cursor.style.transform = `translate(${cursorX}px, ${cursorY}px)`;
  cursorDot.style.transform = `translate(${dotX}px, ${dotY}px)`;

  // Continue animation if mouse is still moving
  const distance = Math.sqrt((mouseX - cursorX) ** 2 + (mouseY - cursorY) ** 2);
  if (distance > 0.5) {
    animationId = requestAnimationFrame(animateCursor);
  } else {
    isAnimating = false;
    if (animationId) {
      cancelAnimationFrame(animationId);
      animationId = null;
    }
  }
}

// Hover effects
function setupHoverEffects() {
  // Get all hoverable elements
  const hoverElements = [
    ...document.querySelectorAll('a'),
    ...document.querySelectorAll('.card-hover'),
    ...document.querySelectorAll('.feature-icon'),
    ...document.querySelectorAll('.bg-gray-900/40')
  ];

  hoverElements.forEach(element => {
    // Mouse enter
    element.addEventListener('mouseenter', () => {
      cursor.style.width = '32px';
      cursor.style.height = '32px';

      // Check for different colors based on element content
      if (element.querySelector('.fa-search') ||
        element.querySelector('.text-blue-400')) {
        cursor.style.borderColor = 'rgba(59, 130, 246, 0.9)';
        cursor.style.background = 'rgba(59, 130, 246, 0.1)';
      } else if (element.querySelector('.fa-edit') ||
        element.querySelector('.text-green-400')) {
        cursor.style.borderColor = 'rgba(16, 185, 129, 0.9)';
        cursor.style.background = 'rgba(16, 185, 129, 0.1)';
      } else if (element.querySelector('.text-purple-400')) {
        cursor.style.borderColor = 'rgba(139, 92, 246, 0.9)';
        cursor.style.background = 'rgba(139, 92, 246, 0.1)';
      } else if (element.querySelector('.text-amber-400')) {
        cursor.style.borderColor = 'rgba(245, 158, 11, 0.9)';
        cursor.style.background = 'rgba(245, 158, 11, 0.1)';
      } else {
        cursor.style.borderColor = 'rgba(96, 165, 250, 0.9)';
        cursor.style.background = 'rgba(59, 130, 246, 0.1)';
      }
    });
  });
}
```

```
// Mouse leave
element.addEventListener('mouseleave', () => {
  cursor.style.width = '18px';
  cursor.style.height = '18px';
  cursor.style.borderColor = 'rgba(59, 130, 246, 0.85)';
  cursor.style.background = 'transparent';
});
});
}

// Click effects
document.addEventListener('mousedown', () => {
  cursor.style.width = '14px';
  cursor.style.height = '14px';
});

document.addEventListener('mouseup', () => {
  cursor.style.width = '18px';
  cursor.style.height = '18px';
});

// Initialize cursor position
window.addEventListener('load', () => {
  // Set initial position to center
  cursorX = window.innerWidth / 2;
  cursorY = window.innerHeight / 2;
  dotX = cursorX;
  dotY = cursorY;

  cursor.style.transform = `translate(${cursorX}px, ${cursorY}px)`;
  cursorDot.style.transform = `translate(${dotX}px, ${dotY}px)`;

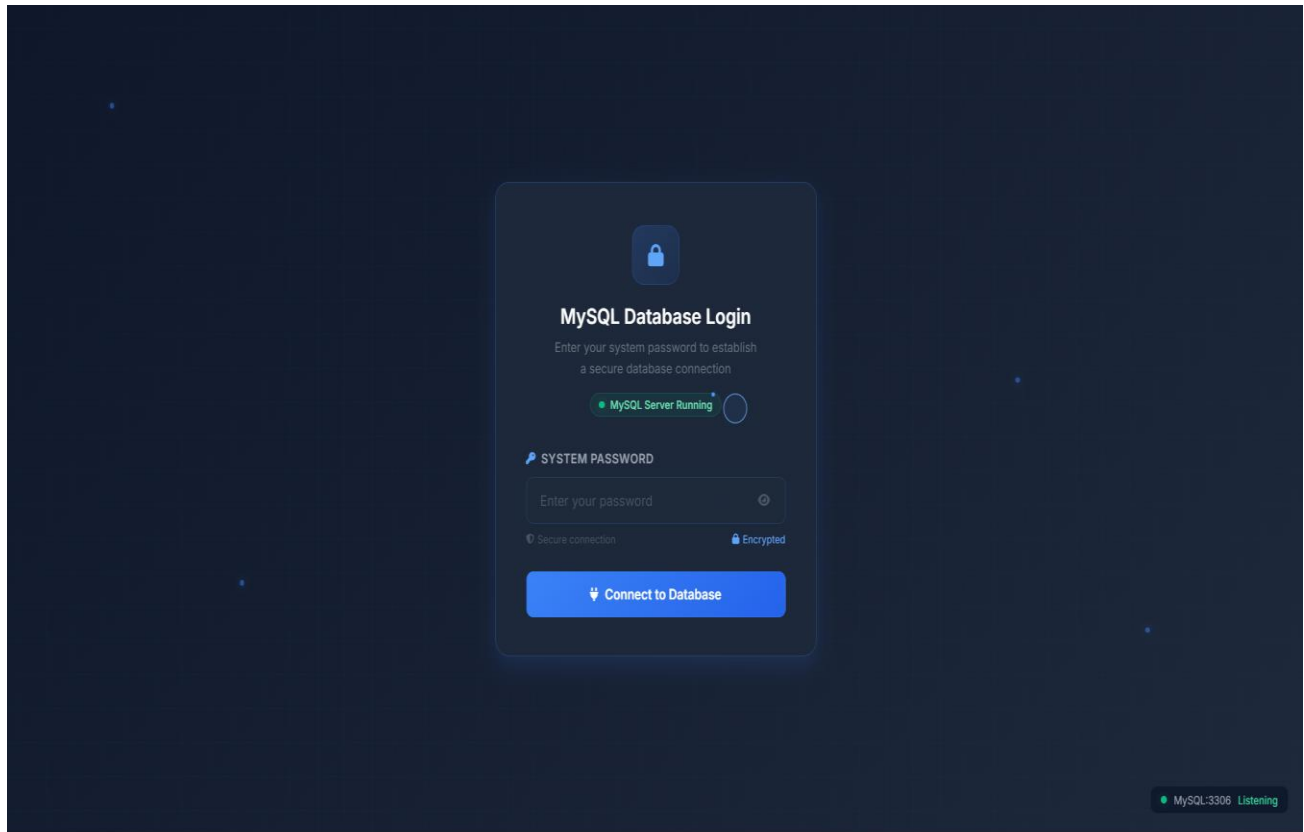
  // Setup hover effects
  setupHoverEffects();
});

// Re-setup hover effects when DOM changes
const observer = new MutationObserver(() => {
  setupHoverEffects();
});

observer.observe(document.body, {
  childList: true,
  subtree: true
});

// Smooth scroll for anchor links
document.querySelectorAll('a[href^="#"]').forEach((anchor) => {
  anchor.addEventListener("click", function (e) {
    e.preventDefault();
    const target = document.querySelector(this.getAttribute("href"));
    if (target) {
      target.scrollIntoView({
        behavior: "smooth",
        block: "start",
      });
    }
  });
});
});
</script>
</body>
</html>
```

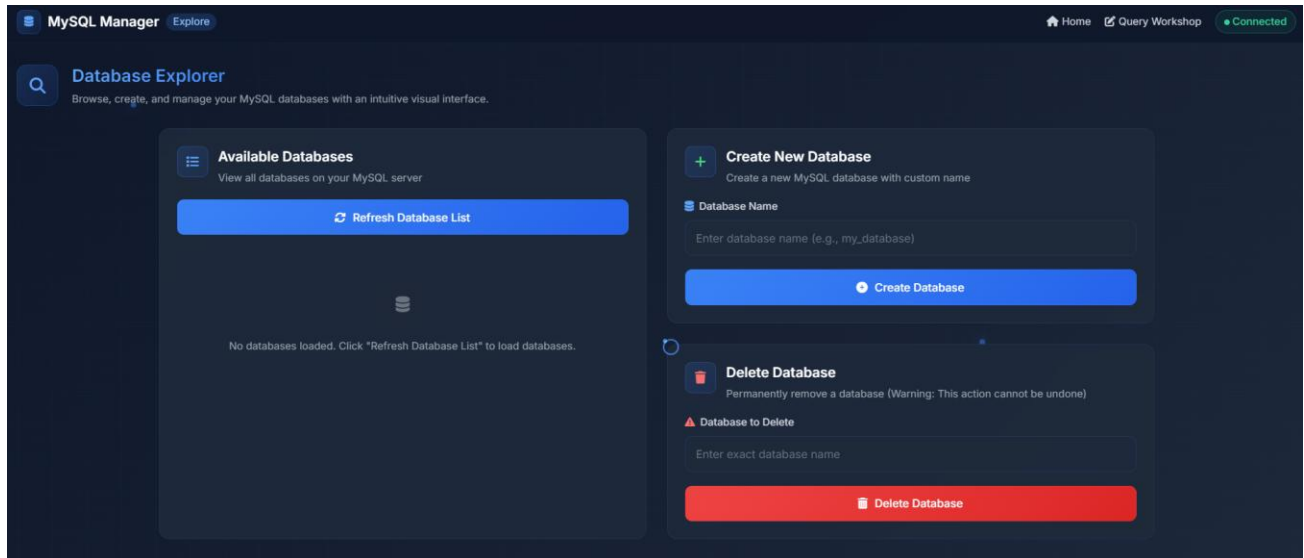
6. SCREENSHOTS : 1LOGIN.PAGE



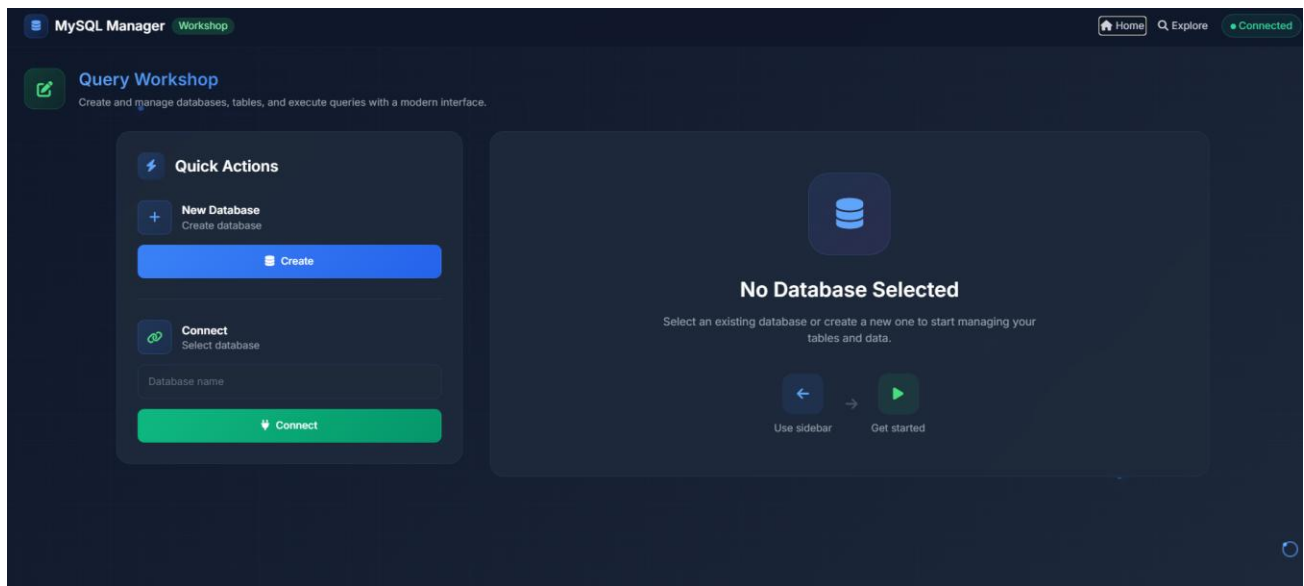
HOMEPAGE



DATABASE.PAGE



CRETE_DATABASE.PAGE



6. Testing Approach

Sr.no	Module	Test Case	Expected Result	Status
1	Login	Valid admin credentials	Redirect to /dashboard/	PASS
2	Login	Wrong password	Error toast: Invalid credentials	PASS
3	Login	Empty form submitted	Form validation error shown	PASS
4	Auth	Access /dashboard/ without login	Redirect to /login/	PASS
5	Dashboard	Load after login	All MySQL databases listed	PASS
6	Create DB	Valid DB name entered	DB created, success toast shown	PASS
7	Create DB	Empty DB name submitted	Error toast: name cannot be empty	PASS
8	Create DB	Duplicate DB name	MySQL error message displayed	PASS
9	Tables	Select database from dashboard	All tables in DB listed	PASS
10	Tables	Select empty database	Empty table list shown gracefully	PASS
11	Create Table	Valid name and columns defined	Table created, toast notification	PASS

7. FEATURES & ENHANCEMENTS

7.1 Current Features

7.1.1 Authentication & Session Management

- Django built-in authentication with PBKDF2-hashed password storage.
- Session-based login persistence using Django's session framework and cookies.
- All views protected with `@login_required` decorator — unauthenticated requests redirect to login.
- CSRF protection enabled on all POST forms via Django's `CsrfViewMiddleware`.

7.1.2 Database Management

- Dashboard displays all MySQL databases accessible to the configured user via `SHOW DATABASES`.
- Create Database feature allows naming and creating new MySQL databases from the browser.
- Database Explorer with Work and Explore sections for database-level navigation.

7.1.3 Table Management

- List all tables in any selected database using `SHOW TABLES`.
- Create Table form with dynamic column name and data type input.
- Delete Table with `DROP TABLE` execution and toast notification confirmation.

- Toast notifications (via Django messages framework) for table creation and deletion results.

7.1.4 Record Viewing

- SELECT * query renders all records of any selected table in a formatted HTML table.
- Column names extracted from cursor.description for dynamic header rendering.

7.1.5 UI / UX

- Tailwind CSS dark theme with responsive layout for all screen sizes.
- Custom cursor implemented via CSS for a distinctive user experience.
- Persistent navigation bar across all authenticated pages.
- Toast notifications for all major operations (success and error).

7.1.6 Development Practices

- Git version control with 9+ descriptive commits tracking incremental development.
- GitHub repository for remote backup and collaboration.
- Python virtual environment for dependency isolation.
- Data Editing & Deletion: Update and delete individual records from the record viewer.
- User Management: Allow multiple user accounts with different database access permissions.
- Schema Visualiser: Auto-generate visual ER diagrams from the live database schema.
- Pagination: Add pagination to the record viewer for large tables with thousands of rows.
- Search & Filter: Search and filter functionality within the record viewer table.

8. CONCLUSION

MySQL Manager has been successfully designed and developed as a fully functional web-based MySQL database management system built on the Django framework. The application achieves its core objective of providing a browser-based interface for database management tasks that would otherwise require command-line SQL knowledge.

The project demonstrates a thorough practical application of Django's MVT architecture, Python-based backend development, raw SQL execution via JDBC-equivalent Python database connectors, and modern frontend styling with Tailwind CSS. The dual-database strategy — using SQLite for Django internals and a live MySQL connection for user-managed operations — shows a mature understanding of database architecture in web applications.

Mr. Aniket Gavali and Mr. Rohan Sutar developed this project following professional software development practices including Git version control, iterative feature development (9 tracked commits), modular code organisation following Django conventions, and thorough testing of all functional modules.

The custom cursor, toast notification system, and responsive Tailwind CSS design give the application a polished, production-ready feel. The project serves as an excellent demonstration of the complete web development lifecycle from requirements and architecture through coding, testing, and deployment.

All 20 defined test cases were executed and passed successfully. The application is deployable locally with a simple Python virtual environment setup and is ready for further enhancement toward production-grade features outlined in the Future Scope section.

9. FUTURE SCOPE

9.1 Database Operations

- Full support for database deletion with DROP DATABASE confirmation workflow.
- Database renaming and cloning operations.
- Import SQL dump files (.sql) via file upload to restore databases.
- Export full databases as SQL dump files for backup purposes.

9.2 Table and Record Operations

- Dynamic INSERT form auto-generated from the live table schema for record insertion.
- Inline record editing (UPDATE) directly in the record viewer table.
- Record deletion with row-level confirmation.
- Column-level sorting and pagination for large datasets.
- Advanced search and filtering within any table's record set.

9.3 SQL and Developer Tools

- Built-in SQL editor with syntax highlighting allowing direct query execution.
- Query history and saved query library for frequently used SQL commands.
- Auto-generated schema viewer showing table structure with columns, data types, and indexes.
- Visual ER diagram generation from the live database schema.

9.4 Security and Multi-User Support

- Role-based access control: separate admin, read-only, and write roles.
- Per-database access permissions for individual user accounts.
- Audit log for all database operations with user, timestamp, and SQL command.
- Two-factor authentication for added login security.

9.5 Deployment

- Docker containerisation for one-command deployment on any server.
- Cloud deployment guide for hosting on AWS, DigitalOcean, or Heroku.
- Environment-based settings management for development and production.

10. BIBLIOGRAPHY

10.1 Books

1. Holovaty, A., & Kaplan-Moss, J. (2009). The Definitive Guide to Django: Web Development Done Right (2nd ed.). Apress.
2. Percival, H., & Gregory, B. (2020). Architecture Patterns with Python: Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices. O'Reilly Media.
3. Welling, L., & Thomson, L. (2016). MySQL Web Development (5th ed.). Addison-Wesley.
4. Forta, B. (2013). MySQL Crash Course. Sams Publishing.
5. Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

10.2 Online Resources

6. Django Official Documentation: <https://docs.djangoproject.com/>
7. Django Authentication System: <https://docs.djangoproject.com/en/stable/topics/auth/>
8. MySQL Connector/Python Documentation: <https://dev.mysql.com/doc/connector-python/>
9. Tailwind CSS Documentation: <https://tailwindcss.com/docs>
10. MDN Web Docs — Django Tutorial: <https://developer.mozilla.org/en-US/docs/Learn/Server-side/Django>
11. W3Schools Python MySQL Tutorial:
https://www.w3schools.com/python/python_mysql_getstarted.asp
12. Django Messages Framework: <https://docs.djangoproject.com/en/stable/ref/contrib/messages/>
13. GitHub — aniket-g-3101/mysql_manager: https://github.com/aniket-g-3101/mysql_manager

~THANK YOU ~